PPE VISION · ARCHITECTURE & INTEGRATION MAP

Mapping Alur Seluruh Sistem

Panduan visual tentang siapa memanggil siapa, data apa yang berpindah, endpoint dan hosting yang digunakan, alur deployment, serta penjelasan jujur tentang peran Ollama dalam AI Summary.

2	1	3
jalur inference	AI lokal	runtime utama

Portal: Laravel 12 · InfinityFree

Inference online: Streamlit Cloud · Python/YOLO

AI summary: Ollama llama3.2:3b · lokal

Tanggal audit: 15 Oktober 2026

JAWABAN PALING PENTING

InfinityFree bukan API, dan Ollama memang hybrid

InfinityFree itu apa?

InfinityFree adalah **hosting** untuk menjalankan Laravel/PHP dan MySQL. Laravel tidak “mengambil API InfinityFree”. Browser mengirim request HTTPS ke Laravel yang kebetulan berjalan di server InfinityFree.

Ollama itu apa?

Ollama adalah runtime model bahasa lokal. Laravel lokal memanggil HTTP API Ollama di `127.0.0.1:11434/api/generate`.

Apakah Ollama dipakai 100%?

Tidak untuk seluruh isi halaman, dan itu disengaja. Angka, persentase, F1, daftar metrik per kelas, serta kelas terlemah dihitung oleh Laravel dari database. Ollama menghasilkan bagian kualitatif: risiko operasional dan tiga rekomendasi. Laravel kemudian memvalidasi dan menyatukan keduanya.

Bagian summary	Pembuat	Alasan
Precision, recall, F1, mAP	Evaluasi YOLO → database → Laravel	Harus presisi dan dapat diuji.
Kelas terlemah	Laravel mengurutkan mAP per kelas	Tidak perlu ditebak LLM.
Risiko operasional	Ollama, lalu divalidasi	Memerlukan penyusunan bahasa dan konteks.
Tiga rekomendasi	Ollama, lalu dinormalisasi Laravel	AI memberi insight tindakan.
Fallback	Rule-based Laravel	Halaman tidak gagal ketika Ollama mati.

Apakah memenuhi assessment? Ya. Ketentuan point plus meminta AI lokal untuk merangkum hasil metrik menggunakan Ollama/llama.cpp. Ollama benar-benar dipanggil dan menyusun bagian analisis. Ketentuan tidak meminta LLM menghitung ulang angka atau menjadi satu-satunya komponen.

Yang harus didemonstrasikan: jalankan Laravel dan Ollama secara lokal, klik “Buat ulang dengan Ollama”, lalu tunjukkan label *Analisis oleh Ollama · llama3.2:3b*. Halaman online menampilkan hasil yang sudah disimpan; ia tidak memanggil laptop secara live.

DAFTAR ISI

Peta pembahasan

1. Daftar komponen
2. Alur production online
3. Alur Dashboard
4. Alur Demo Streamlit
5. Alur FastAPI lokal
6. Alur AI Summary Ollama
7. Mengapa hybrid, bukan 100% LLM
8. Alur training dan artefak
9. Alur deployment
10. Konfigurasi lokal vs production
11. Endpoint, port, payload
12. Keamanan dan trust boundary
13. Mapping ke assessment
14. Demo integrasi
15. Tanya-jawab

BAB 1

Siapa melakukan apa?

Komponen	Runtime/lokasi	Tanggung jawab	Tidak melakukan
Browser	Perangkat pengguna	Meminta halaman, menampilkan iframe, upload ke Streamlit.	Tidak menghitung metrik.
Laravel	PHP · InfinityFree/lokal	Routing, UI, database, validasi, summary orchestration.	Tidak menjalankan bobot YOLO production.
MySQL	InfinityFree	Menyimpan eksperimen, metrik, narasi summary.	Tidak memanggil AI.
Streamlit	Python · Community Cloud	UI demo dan inference gambar/video/kamera.	Tidak menyimpan dashboard Laravel.
Ultralytics YOLO	Di proses Python	Memuat `best.pt`, menghasilkan box/class/confidence.	Tidak membuat AI Summary.
FastAPI	Python lokal, port 8001	Alternatif REST API inference untuk Laravel lokal.	Tidak aktif di InfinityFree.
Ollama	Lokal, port 11434	Menjalankan llama3.2:3b dan memberi analisis kualitatif.	Tidak dihitung sebagai sumber angka.
Colab	Google GPU	Training, random search, evaluasi, menghasilkan artefak.	Tidak menjadi web production.
GitHub	Repository	Sumber deploy Streamlit dan version control.	Tidak menyimpan secret.
FileZilla	Komputer lokal	Mengunggah paket Laravel ke InfinityFree.	Bukan runtime aplikasi.

> **Tiga runtime berbeda:** PHP menangani portal, Python menangani computer vision, dan Ollama menjalankan LLM lokal. Mereka terhubung lewat HTTP atau data yang disimpan, bukan berada di satu proses.

BAB 2

Arsitektur production yang aktif



Urutan request halaman biasa

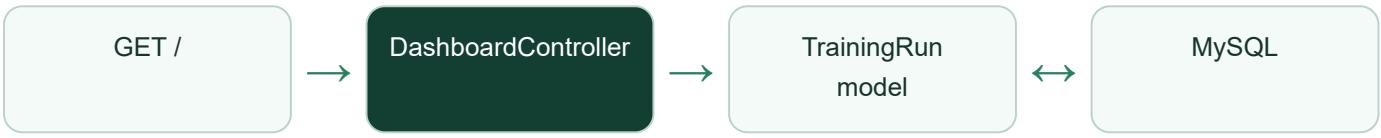
Contoh: pengguna membuka Dashboard

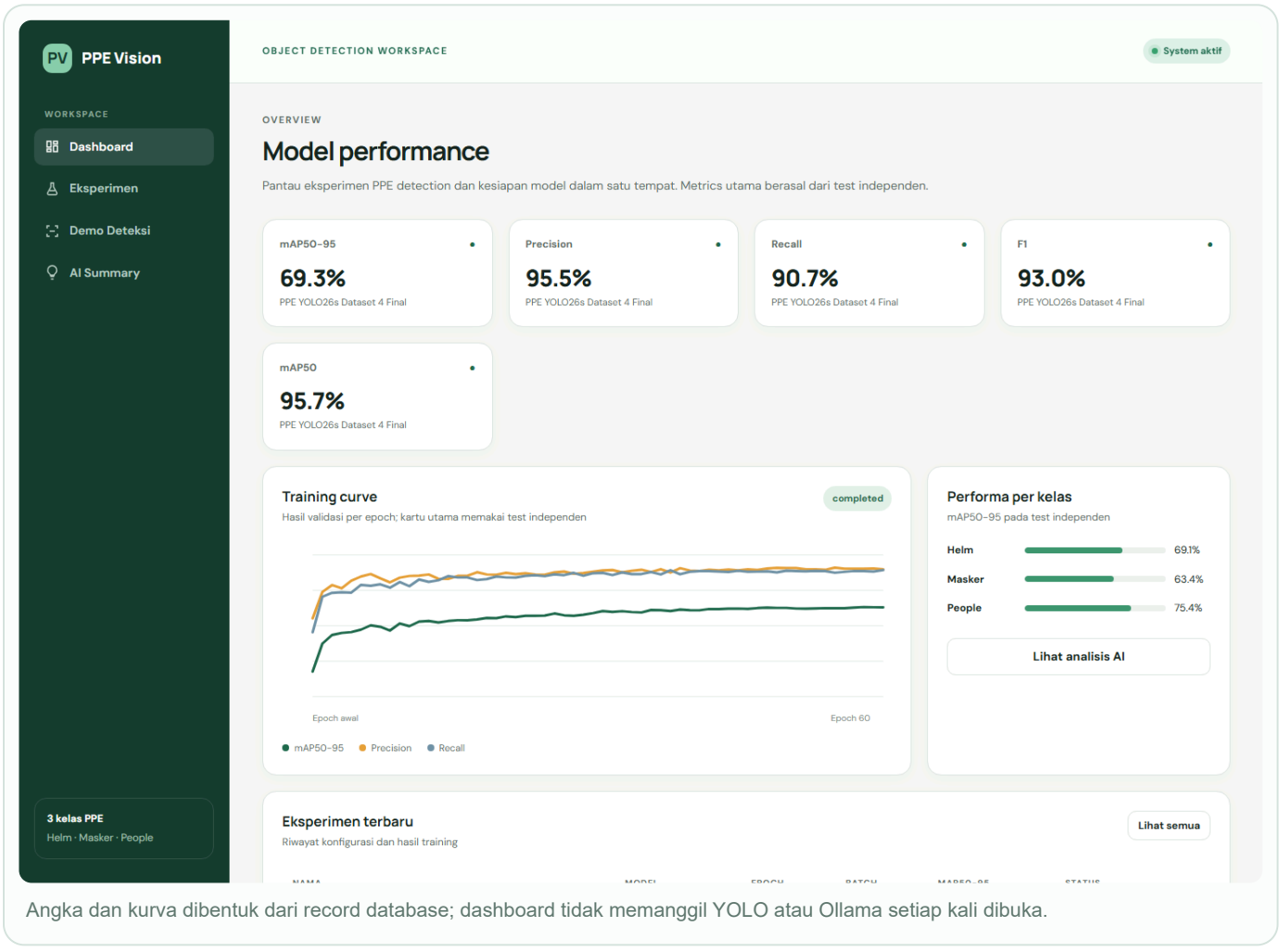
1	**Browser**	GET `https://ppe-vision.infinityfreeapp.com/`.
2	**Web server**	Menjalankan `public/index.php` Laravel.
3	**Controller**	Mengambil latest run dan lima eksperimen dari MySQL.
4	**Blade**	Membentuk HTML kartu, grafik SVG, tabel, dan navigasi.
5	**Browser**	Menerima HTML/CSS/JS dan merender halaman.

Tidak ada "API InfinityFree". InfinityFree menyediakan server PHP, database host, file storage, dan alamat domain. API aplikasi adalah route Laravel sendiri, API Streamlit internal, FastAPI lokal, dan API Ollama lokal.

BAB 3

Mapping alur Dashboard dan Eksperimen





Angka dan kurva dibentuk dari record database; dashboard tidak memanggil YOLO atau Ollama setiap kali dibuka.

Bagian layar	Sumber data	Transformasi
Lima kartu	`training_runs.metrics`	Nilai 0–1 dikali 100 dan diformat satu desimal.
Training curve	`training_history` JSON	Titik per epoch menjadi SVG polyline.
Per kelas	`per_class_metrics`	mAP50-95 menjadi panjang bar.
Eksperimen terbaru	Lima `TrainingRun` terbaru	Ditampilkan sebagai tabel.
AI Summary	`ai_summary` tersimpan	Markdown aman menjadi HTML.

Eksperimen bersifat read-only

Versi final mempertahankan daftar dan detail eksperimen sebagai bukti setting, metrik, serta artefak training. Fitur membuat eksperimen baru dihapus karena tidak menjalankan training dan tidak diminta assessment; training tetap

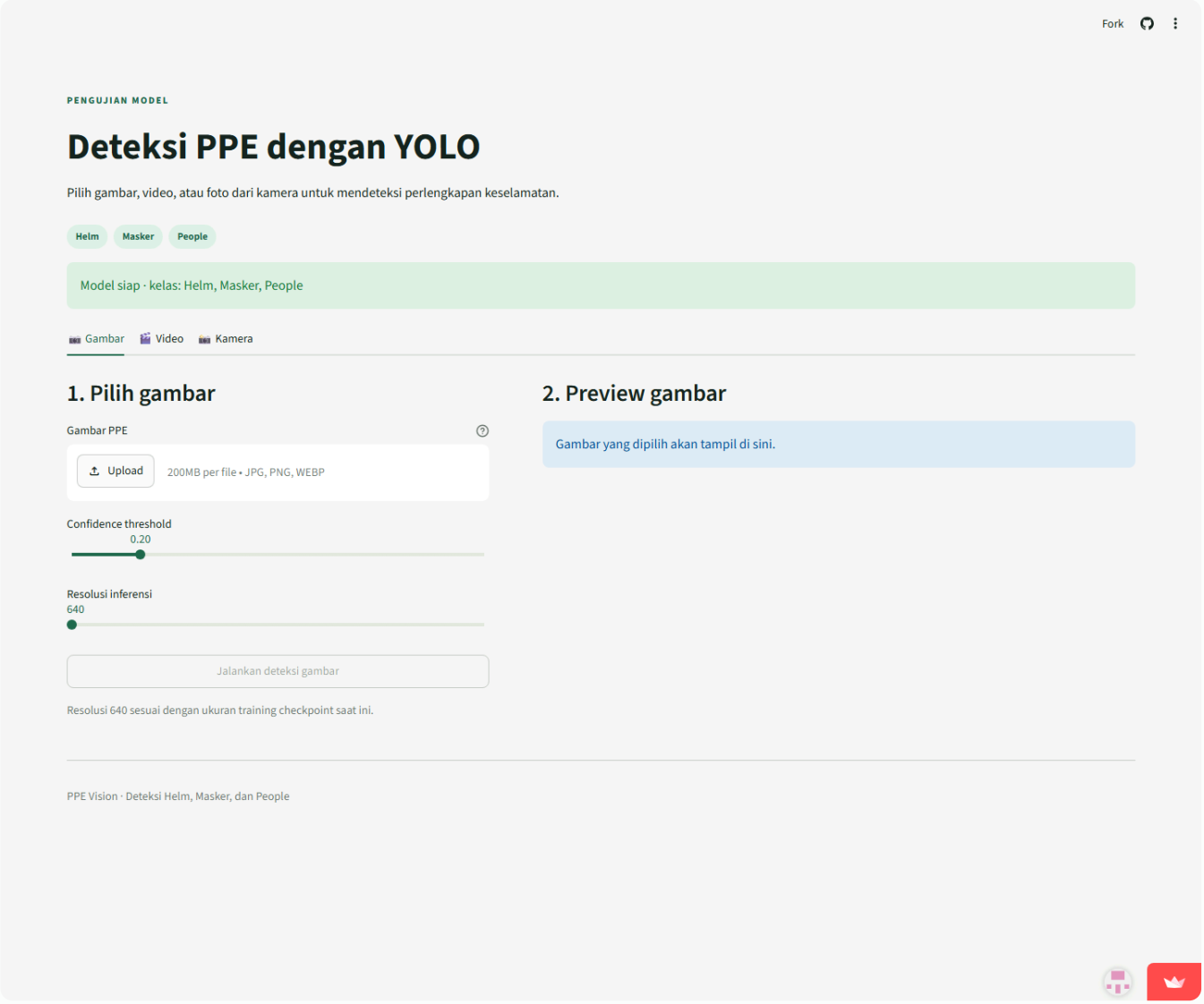
dilakukan di Google Colab.

BAB 4

Mapping alur Demo Streamlit online

Jalur production aktif

1	**Browser**	Meminta `/detection` ke Laravel.
2	**Laravel**	Membaca `STREAMLIT_DEMO_URL` dan menghasilkan URL `?embed=true`.
3	**Blade**	Mengirim elemen `<iframe>` kepada browser.
4	**Browser**	Browser—bukan server InfinityFree—meminta aplikasi langsung ke Streamlit Cloud.
5	**Streamlit**	Memuat Python dan checkpoint `best.pt`, lalu menampilkan “Model siap”.
6	**Pengguna**	Mengunggah gambar/video atau mengambil foto kamera ke Streamlit.
7	**YOLO**	`model.predict()` menghasilkan box, class, confidence, dan waktu inferensi.
8	**Streamlit**	Menggambar hasil dan mengirim tampilan kembali ke browser.



Streamlit adalah aplikasi Python terpisah yang ditanam dalam portal Laravel.

Kenapa browser yang menghubungi Streamlit?

Iframe bekerja client-side. InfinityFree hanya mengirim alamat iframe dalam HTML. Ini menghindari kebutuhan PHP server untuk meneruskan video dan menjalankan Python.

> **Cold start:** bila Streamlit tidur, browser melihat “waking up”. Setelah container, dependency, dan model dimuat, demo kembali siap tanpa deploy ulang.

BAB 5

Mapping FastAPI: jalur lokal alternatif

Repository juga memiliki jalur API inference untuk integrasi server-to-server. Jalur ini berguna ketika Laravel dan Python berjalan di komputer/server yang dapat saling menjangkau.



Urutan lokal

1	**Health check**	Laravel GET `YOLO_API_URL/health`; harus mengembalikan `status=ready`.
2	**Upload**	Laravel memvalidasi JPG/PNG/WEBP maksimum 10 MB dan confidence 0,05–0,95.
3	**API call**	`YoloInferenceService` POST multipart ke `/predict`.
4	**Inference**	FastAPI memvalidasi file, membuka image, dan memanggil YOLO.
5	**Response**	JSON berisi detections, inference_ms, dan annotated_image base64.
6	**Persist**	Laravel menyimpan hasil gambar dan history detection.

```
GET  http://127.0.0.1:8001/health
POST http://127.0.0.1:8001/predict

Response:
{
  "detections": [{"class":"helm","confidence":0.91,"bbox":[...]}],
  "inference_ms": 193.38,
  "annotated_image": "...base64..."
}
```

> **Mengapa tidak aktif production?** InfinityFree tidak menjalankan Python/FastAPI dan tidak dapat mengakses `127.0.0.1:8001` pada laptop. Production memakai iframe Streamlit sebagai jalur yang tersedia.

BAB 6

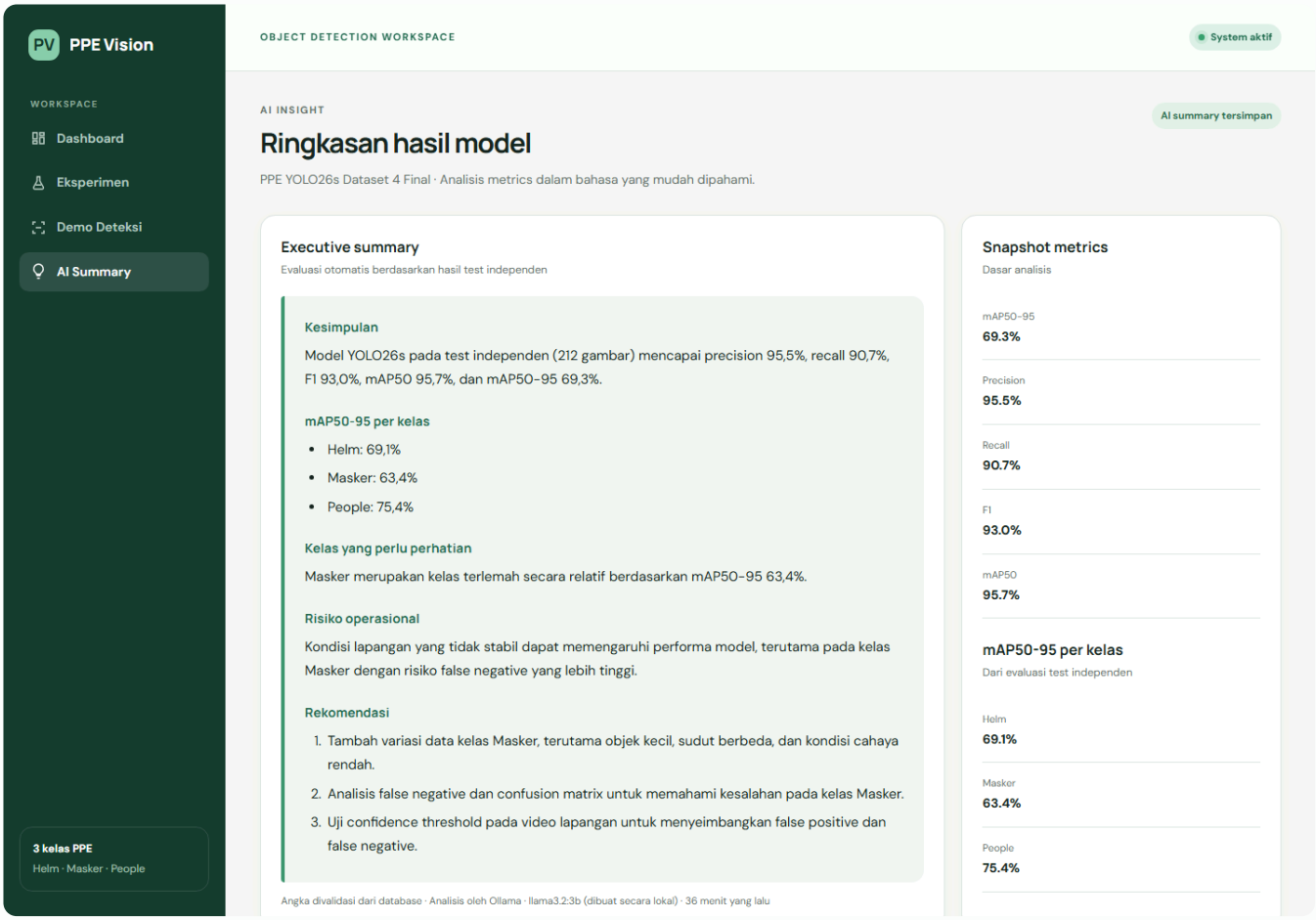
Mapping AI Summary dengan Ollama lokal



Urutan generate

1	**Pengguna**	Klik “Buat ulang dengan Ollama” pada Laravel lokal.
2	**Controller**	Memastikan operasi admin aktif dan metrik tersedia.
3	**Service**	Mengambil model setting, split evaluasi, overall metrics, per-class metrics, dan kelas terlemah.
4	**HTTP API**	POST ke `http://127.0.0.1:11434/api/generate` dengan model, prompt, schema, `stream=false`.
5	**Ollama**	Menjalankan llama3.2:3b dan mengembalikan JSON.
6	**Validasi**	Laravel memeriksa `operational_risk` dan tepat tiga recommendations.
7	**Normalisasi**	Laravel memastikan fokus pada kelas terlemah, confusion matrix, dan confidence threshold.
8	**Gabung**	Angka deterministic + narasi AI menjadi Markdown terstruktur.
9	**Simpan**	`ai_summary` dan `summary_generated_by` disimpan ke database.

```
POST /api/generate
{
  "model": "llama3.2:3b",
  "stream": false,
  "prompt": "...data metrics terstruktur...",
  "format": {
    "required": ["operational_risk", "recommendations"]
  },
  "options": {"temperature": 0.2, "num_predict": 360}
}
```



Production membaca summary yang sudah tersimpan. Ia tidak menghubungi Ollama lokal setiap page load.

BAB 7

Mengapa tidak 100% diserahkan kepada Ollama?

1. Angka tidak boleh dihalusinasikan

LLM dirancang menghasilkan bahasa, bukan menjadi kalkulator sumber kebenaran. Bila prompt berisi 69,299%, model dapat membulatkan, menukar angka, atau menulis metrik yang tidak tersedia. Laravel dapat memformat nilai database secara pasti.

2. Kelas terlemah dapat dihitung deterministik

Mengurutkan tiga nilai mAP adalah operasi sederhana. Menyerahkannya kepada LLM menambah ketidakpastian tanpa manfaat.

3. Halaman harus tetap tersedia

Model lokal pertama kali dimuat bisa lambat, Ollama bisa belum dijalankan, atau request dapat timeout. Fallback membuat pengguna tetap memperoleh ringkasan dasar dan label engine secara jujur.

4. AI tetap benar-benar digunakan

Ollama menyusun risiko dan rekomendasi—bagian yang memerlukan bahasa serta pertimbangan kualitatif. Laravel bukan mengganti hasil Ollama secara total; Laravel menjaga batas agar output relevan dan aman.

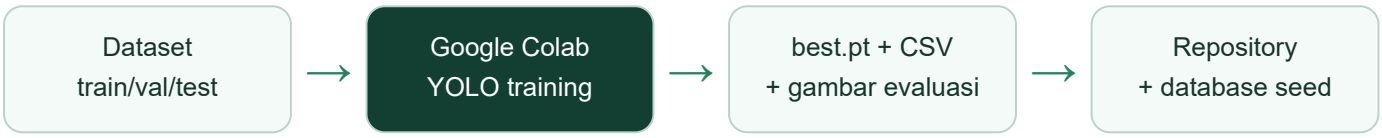
Desain	Kelebihan	Risiko
100% LLM	Terlihat “lebih AI”.	Angka bisa salah, format berubah, gagal jika model mati.
Hybrid sekarang	Angka akurat, AI memberi insight, fallback tersedia.	Perlu menjelaskan pembagian tanggung jawab.
100% rule-based	Stabil dan cepat.	Tidak membuktikan penggunaan Ollama.

Kesimpulan assessment: hybrid adalah penggunaan Ollama yang valid dan secara engineering lebih baik. Yang penting, saat engine berhasil, metadata harus tertulis “Ollama · llama3.2:3b”; saat fallback, aplikasi harus jujur menulis “Analisis lokal berbasis aturan (bukan LLM)”.

Jika assessor meminta “harus sepenuhnya Ollama”: jangan biarkan LLM membuat angka. Jelaskan bahwa seluruh narasi kualitatif dibuat Ollama, sementara angka dikunci dari hasil evaluasi. Ini adalah kontrol integritas, bukan pengurangan AI.

BAB 8

Mapping training sampai tampil di web



Alur data ML

1	**Dataset**	3.025 train, 603 validation, 212 test; kelas Helmet, Mask, Person.
2	**Random search**	Lima trial × 20 epoch pada subset; trial 4 dipilih.
3	**Final training**	YOLO26s, 60 epoch, batch 16, 640 px, AdamW.
4	**Checkpoint**	Best epoch 58 menghasilkan `best.pt`.
5	**Evaluasi**	Test menghasilkan precision, recall, F1, mAP50, mAP50-95, per-class metrics.
6	**Artefak**	`results.csv`, confusion matrix, PR/F1/P/R curves, labels, contoh prediksi.
7	**Web data**	Seeder menyinkronkan metadata dan path artefak ke `training_runs`.
8	**Inference**	Checkpoint yang sama digunakan oleh Streamlit/FastAPI.

Mengapa model tidak ada di InfinityFree? PHP portal tidak menjalankan inference. Paket build sengaja tidak menyalin backup checkpoint ke hosting PHP; checkpoint digunakan di runtime Python Streamlit.

BAB 9

Mapping deployment

A. Laravel ke InfinityFree



1. `build-package.ps1` menyalin source PHP.
2. Composer memasang dependency production dan mengoptimalkan autoloader.
3. npm membangun asset Vite.
4. Paket update-safe dibuat tanpa `.env`.
5. FileZilla mengunggah isi `htdocs`.
6. Laravel menggunakan kredensial MySQL dari control panel.

B. Streamlit dari GitHub



1. Streamlit Cloud membaca repository, branch, dan `streamlit_app/app.py`.
2. Dependency dipasang dari `streamlit_app/requirements.txt`.
3. Model bundled diverifikasi checksum jika konfigurasi checksum tersedia.
4. Perubahan membutuhkan push/redeploy Streamlit; data input pengguna tidak membutuhkan deploy.

C. Summary lokal ke online

Ollama menghasilkan summary pada database lokal. Agar production menampilkan versi yang sama, teks final tersedia di seeder/record database production. Production hanya membaca dan tidak memanggil Ollama.

BAB 10

Konfigurasi lokal vs production

Variabel	Lokal	InfinityFree	Makna
DB_CONNECTION	sqlite	mysql	Lokasi penyimpanan Laravel.
YOLO_API_URL	127.0.0.1:8001 opsional	Kosong	FastAPI hanya jalur lokal.
STREAMLIT_DEMO_URL	Opsional	URL Streamlit publik	Menentukan iframe demo.
OLLAMA_ENABLED	true saat generate	false	Ollama hanya dapat diakses lokal.
OLLAMA_URL	127.0.0.1:11434	Tidak dapat menjangkau laptop	Endpoint API model lokal.
ALLOW_ADMIN_WRITES	true	false	Production read-only.
APP_DEBUG	true	false	Error sensitif tidak tampil online.

Mengapa 127.0.0.1 berbeda?

Laravel lokal

127.0.0.1 menunjuk laptop yang sama. Laravel dapat menjangkau Ollama dan FastAPI.

Laravel InfinityFree

127.0.0.1 menunjuk server InfinityFree. Ia tidak pernah berarti laptop Anda.

Jangan membuka Ollama mentah ke internet. API lokal umumnya tidak memiliki autentikasi aplikasi. Jika suatu hari dibuat remote, letakkan di belakang HTTPS, auth, firewall, rate limit, dan logging.

BAB 11

Endpoint, port, dan data yang lewat

Dari → ke	Alamat	Metode/data	Aktif di
Browser → Laravel	HTTPS port 443	GET halaman, POST form	Lokal + production.
Laravel → MySQL	Host InfinityFree:3306	SQL melalui PDO	Production.
Browser → Streamlit	HTTPS port 443	Iframe, upload, WebSocket	Production.
Laravel → FastAPI	`127.0.0.1:8001`	GET /health, POST /predict multipart	Lokal alternatif.
Laravel → Ollama	`127.0.0.1:11434`	POST /api/generate JSON	Lokal.
Browser → Laravel dev	`127.0.0.1:8000`	HTTP lokal	Development.
GitHub → Streamlit	Platform integration	Source checkout/build	Deployment.
FileZilla → FTP host	FTP/FTPS	File package	Deployment Laravel.

Yang bukan API runtime

- FileZilla: hanya alat transfer.
- GitHub: source/deployment trigger, bukan endpoint inference.
- InfinityFree control panel: konfigurasi hosting, bukan API aplikasi.
- Colab: lingkungan training, bukan service production.

BAB 12

Trust boundary dan keamanan alur

Batas	Risiko	Kontrol
Browser → Laravel	Input berbahaya, CSRF, spam	Validation, CSRF, throttle.
Browser → Streamlit	File besar/palsu, kamera	Type check, Pillow safety, izin kamera, batas hosting.
Laravel → FastAPI	Service tidak siap, timeout	/health, timeout, MIME/size validation.
Laravel → Ollama	Timeout, format salah, halusinasi	Connect timeout, 180s timeout, JSON schema, fallback.
Laravel → MySQL	Credential bocor	`.env` tidak di-Git, update-safe tidak membawa .env.
Public admin routes	Data diubah pengunjung	`ALLOW_ADMIN_WRITES=false`, route 404.
Summary → HTML	XSS dari Markdown	Strip HTML dan unsafe links.

CORS Streamlit final

Origin yang diizinkan kini hanya localhost development dan domain InfinityFree production. Origin Render dan tunnel Cloudflare lama telah dibersihkan.

```
corsAllowedOrigins = [
  "http://127.0.0.1:8000",
  "http://localhost:8000",
  "https://ppe-vision.infinityfreeapp.com",
]
```

BAB 13

Mapping arsitektur ke ketentuan assessment

Ketentuan	Komponen yang membuktikan	Alur
YOLO + Python	Streamlit/FastAPI + best.pt	Input → model.predict → box/class/confidence.
Helm, Masker, People	Model metadata + mapping label	Helmet/Mask/Person → helm/masker/people.
Metrik training	Colab → CSV/artefak → DB → Dashboard	Evaluasi → seed/import → visual.
Summary menggunakan AI	Laravel → Ollama lokal → validasi → DB	Metrik deterministic + insight AI.
Web setting/hasil/demo	Laravel detail + Streamlit iframe	DB → Blade; browser → Streamlit.
Point plus Laravel	Route/controller/model/Blade/MySQL	HTTP request → MVC → response.
Point plus AI lokal	Ollama llama3.2:3b	POST /api/generate lokal.
Online hosting	InfinityFree + Streamlit Cloud	Browser dapat mengakses keduanya via HTTPS.

Status Ollama: memenuhi, dengan syarat Anda menunjukkan bukti run lokal. Fallback tidak membatalkan pemenuhan; fallback adalah mekanisme ketahanan. Metadata engine membedakan hasil Ollama dari hasil rule-based.

BAB 14

Demo integrasi yang meyakinkan

Sebelum presentasi lokal

```
ollama serve
ollama list

php artisan serve
# Pastikan pada .env lokal:
OLLAMA_ENABLED=true
OLLAMA_URL=http://127.0.0.1:11434
OLLAMA_MODEL=llama3.2:3b
```

Urutan demo 6 menit

1. **Portal online:** buka dashboard InfinityFree dan jelaskan hosting PHP/MySQL.
2. **Demo:** buka /detection; jelaskan browser memuat Streamlit iframe secara langsung.
3. **YOLO:** uji gambar; tunjukkan bounding box dan tiga kelas.
4. **Metrics:** kembali ke dashboard/detail dan tunjukkan data test/artefak.
5. **AI lokal:** buka Laravel lokal → AI Summary → “Buat ulang dengan Ollama”.
6. **Bukti engine:** tunjukkan “Analisis oleh Ollama · llama3.2:3b”.
7. **Keamanan:** jelaskan production menampilkan hasil tersimpan dan tidak membuka Ollama.

Kalimat presentasi

> “Arsitektur saya sengaja dipisah. InfinityFree menjalankan Laravel dan MySQL, Streamlit Cloud menjalankan Python/YOLO, dan Ollama menjalankan LLM secara lokal. Laravel mengorkestrasi data dan UI. AI hanya membuat insight kualitatif; angka tetap berasal dari evaluasi model agar tidak berhalusinasi.”

BAB 15

Tanya-jawab integrasi

Q: Laravel ambil API dari InfinityFree?
Tidak. Laravel berjalan di InfinityFree. Ia membaca MySQL menggunakan koneksi database dan mengirim HTML ke browser.

Q: Siapa yang memanggil Streamlit?
Browser pengguna melalui iframe. Server Laravel hanya menyisipkan URL Streamlit.

Q: Apakah Laravel online memanggil Ollama laptop?
Tidak. Alamat localhost pada server berbeda dari localhost laptop. Production membaca summary tersimpan.

Q: Kalau begitu apakah Ollama benar-benar dipakai?
Ya, saat generate lokal Laravel mengirim request ke API Ollama. Hasil disimpan bersama nama engine. Automated test juga memverifikasi bentuk request dan output.

Q: Mengapa ada fallback?
Agar aplikasi tidak gagal ketika model lokal belum aktif atau timeout. Label engine jujur membedakan fallback dari LLM.

Q: Mengapa tidak semua kalimat berasal dari Ollama?
Angka dan peringkat kelas adalah fakta yang harus deterministik. Ollama difokuskan pada risiko dan rekomendasi, tempat kemampuan bahasa memberi nilai.

Q: Apakah assessment mewajibkan 100% LLM?
Tidak. Teks meminta AI lokal untuk summarization menggunakan Ollama/llama.cpp. Desain hybrid tetap memakai Ollama untuk summary sambil mengunci fakta numerik.

Q: FastAPI dipakai online?
Tidak pada deployment final. Ia adalah jalur integrasi lokal alternatif dan bukti Laravel dapat berkomunikasi dengan service Python melalui REST.

Q: Mengapa dua jalur inference?
Streamlit cocok untuk demo hosted gratis; FastAPI cocok untuk arsitektur service-to-service ketika Python server tersedia.

Checklist pemahaman

- □ Dapat menjelaskan InfinityFree adalah hosting, bukan API.
- □ Dapat menggambar jalur browser → Laravel → MySQL.
- □ Dapat menggambar jalur browser → Streamlit → YOLO.
- □ Dapat menjelaskan FastAPI hanya alternatif lokal.
- □ Dapat menjelaskan Laravel → Ollama lokal → DB.
- □ Dapat menjelaskan mengapa summary online tidak live ke Ollama.
- □ Dapat mempertahankan alasan desain hybrid.

Laporan dibuat dari audit konfigurasi, controller, service, view, runtime Python, deployment script, dan test. Secret, password, token, dan APP_KEY tidak disertakan.